Lecture 8: Iterative Optimization & Logistic Regression

COMP90049
Introduction to Machine Learning
Semester 2, 2025

Lea Frermann, CIS

Copyright @ University of Melbourne 2025. All rights reserved. No part of the publication may be reproduced in any form by print, photoprint, microfilm or any other means without written permission from the author.

Iterative Optimization

Finding Optimal Points I

Finding the parameters that optimize a **target**

Ex1: Estimate the study time which leads to the **best grade** in COMP90049.

Ex2: Find the shoe price which leads to **maximum profit** of our shoe shop.

Ex3: Predicting **housing prices** from a weighted combination of house age and house location

Ex4: Find the parameters θ of a spam classifier which lead to the **lowest error**

Ex5: Find the parameters θ of a spam classifier which lead to the **highest data log likelihood**

Recipe for finding Minima / Maxima

1. Define your function of interest $f(x)$ (e.g., data log likelihood)
2. Compute its first derivative wrt its input x
3. Set the derivative to zero
4. Solve for x

Closed-form vs Iterative Optimization

Closed-form solutions

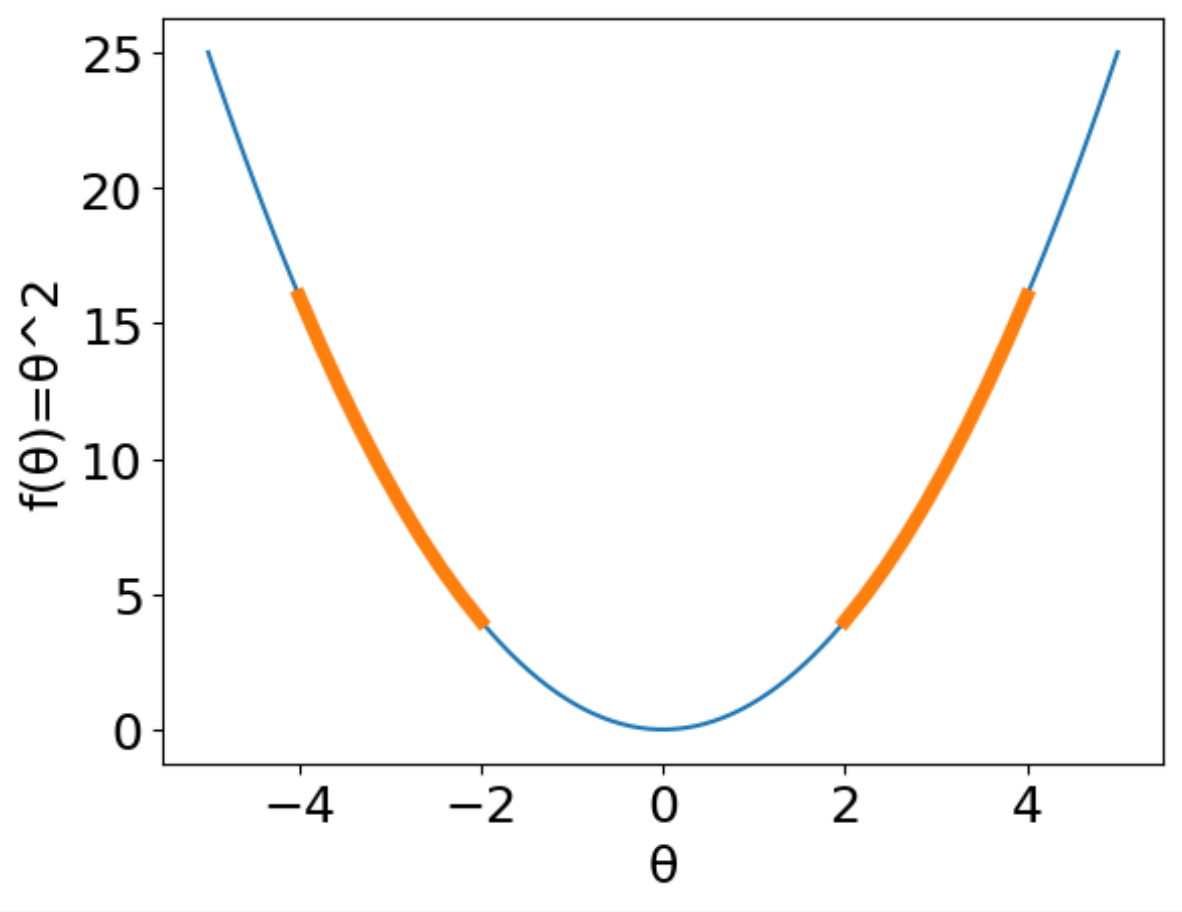
- Previously, we computed the **closed form** solution for the MLE of the binomial distribution
- We follow our recipe, and arrive at a single solution

Unfortunately, life is not always as easy

- Often, no closed-form solution exists
- Instead, we have to **iteratively** improve our estimate of $\hat{\theta}$ until we arrive at a satisfactory solution
- Gradient descent is one popular iterative optimization method

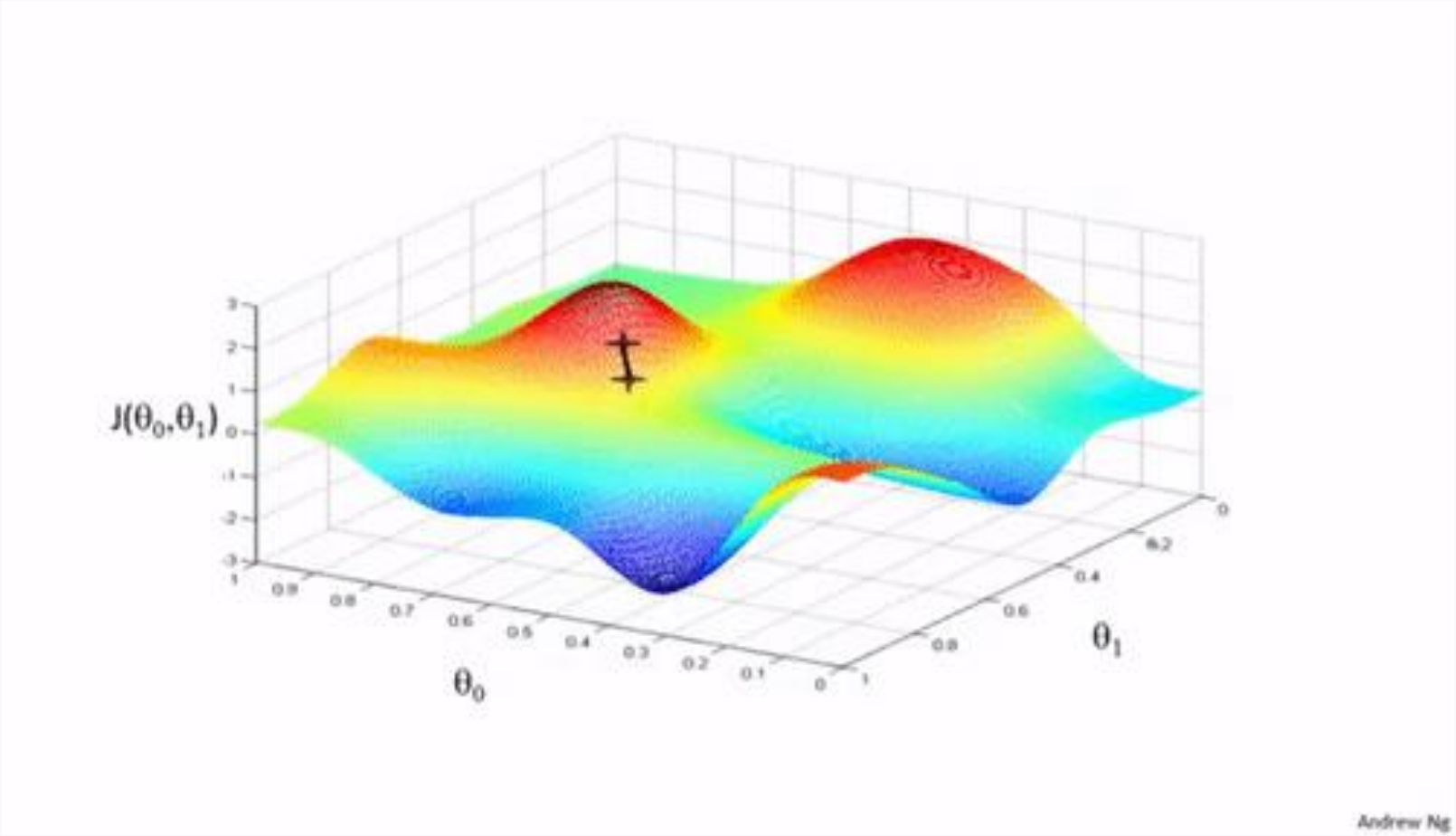


‘Descending’ the function to find the Optimum



- 1-dimensional case: find parameter θ that minimizes the function
- follow the curvature of the line step by step

‘Descending’ the function to find the Optimum



- 2-dimensional case: find parameters $\theta = [\theta_0, \theta_1]$ that minimize the function J
- follow the curvature step by step along the steepest way

Source: https://medium.com/binaryandmore/beginners-guide-to-deriving-and-implementing-backpropagation-e3c1a5a1e536

Gradient Descent: Intuition

Intuition

- Descending a mountain (aka. our function) as fast as possible: at every position take the next step that takes you most directly into the valley
- We compute a series of solutions $\theta^{(0)}, \theta^{(2)}, \theta^{(3)}, \ldots$ by 'walking' along the function and taking steps in the direction with the steepest local slope (or gradient).
- each solution depends on the current location

Gradient Descent: Details

Learn the model parameters θ

- such that we **minimize the error**
- traverse over the loss function step by step (‘descending into a valley’)
- we would like an algorithm that tells how to update our parameters

$$\theta \leftarrow \theta + \Delta\theta$$

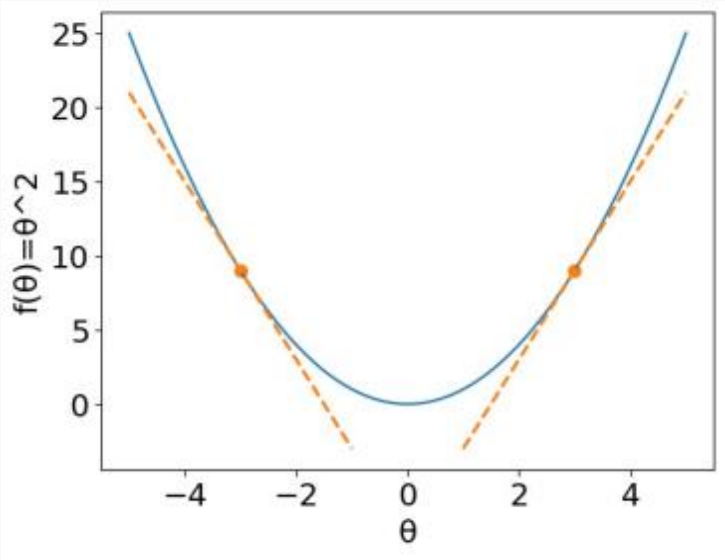
Gradient Descent: Details

Learn the model parameters θ

- such that we **minimise the error**
- traverse over the loss function step by step ('descending into a valley')
- we would like an algorithm that tells us how to update our parameters

$$\theta \leftarrow \theta + \Delta\theta$$

- $\Delta\theta$ is the **derivative** $\frac{\partial f}{\partial \theta}$
- tells us how much *f* changes in response to a change in θ.
- a measure of the **slope** or **gradient** of a function *f* at point θ
- the **gradient** points to the greatest **increase** of a function



Gradient Descent: Details

Learn the model parameters θ

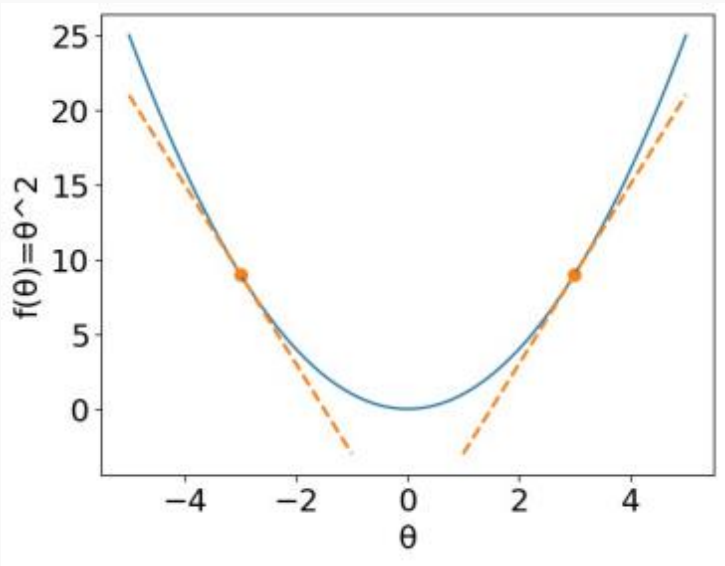
- such that we **minimise the error**
- traverse over the loss function step by step ('descending into a valley')
- we would like an algorithm that tells us how to update our parameters

$$\theta \leftarrow \theta + \Delta\theta$$

- if $\frac{\partial f}{\partial \theta} < 0$: as θ : ↗ $f(\theta)$: ↘
- if $\frac{\partial f}{\partial \theta} > 0$: as θ : ↘ $f(\theta)$: ↘
- if $\frac{\partial f}{\partial \theta} = 0$: we are at a minimum
- so, to approach the minimum:

$$\theta^{t+1} = \theta^t - \eta \frac{\partial f}{\partial \theta^t}$$

Learning Rate



Gradient Descent for multiple parameters

- Usually, our models have **several parameters** which need to be optimized to minimize the error
- We compute **partial derivatives** of $f(\theta)$ wrt. individual θ_i
- Partial derivatives measure change in a function of multiple parameters given a change in a single parameter, with all others held constant
- For example, for $f(\theta_1, \theta_2)$ we can compute $\frac{\partial f}{\partial \theta_1}$ and $\frac{\partial f}{\partial \theta_2}$
- We then **update each parameter individually**

$$\theta_1 \leftarrow \theta_1 + \Delta\theta_1 \qquad \text{with } \Delta\theta_1 = -\eta \frac{\partial f}{\partial \theta_1}$$

$$\theta_2 \leftarrow \theta_2 + \Delta\theta_2 \qquad \text{with } \Delta\theta_2 = -\eta \frac{\partial f}{\partial \theta_2}$$

Gradient Descent: Algorithm

Recipe for Gradient Descent (single parameter)

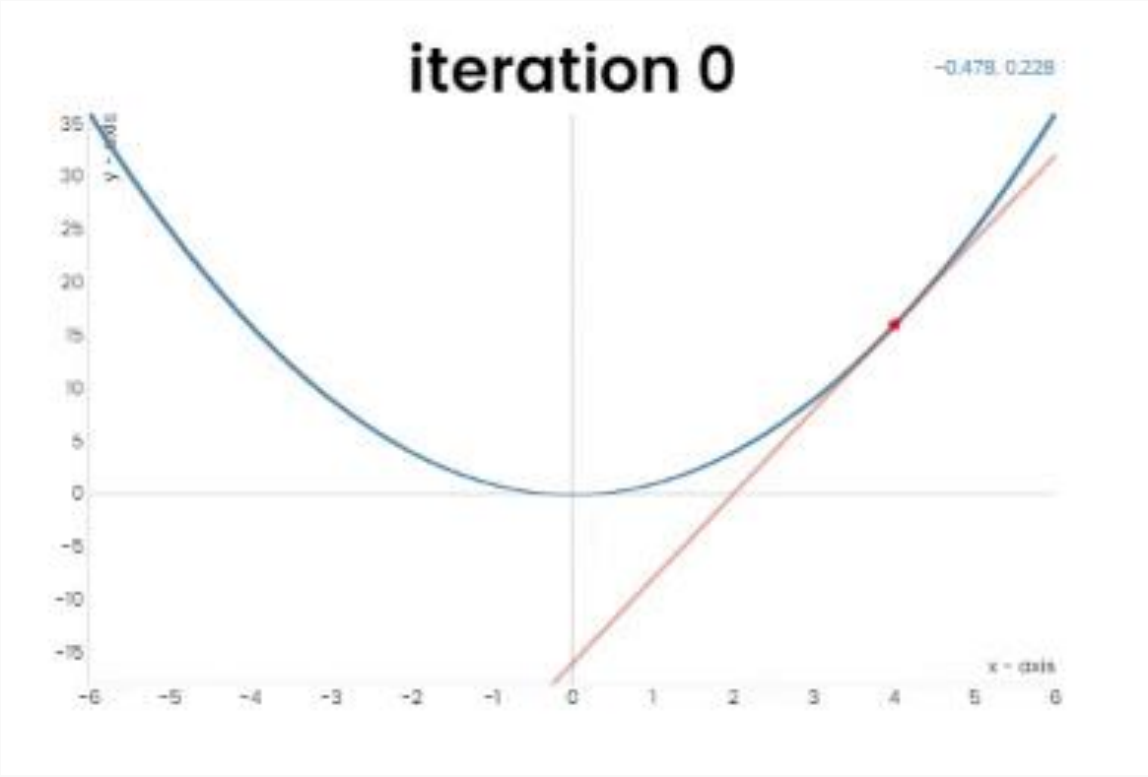
1: Define objective function $f(\theta)$
2: Initialize parameter $\theta^{(0)}$
3: **for** iteration $t \in \{0, 1, 2, ...T\}$ **do**
4: Compute the first derivative of f at that point $\theta^{(t)}$: $\frac{\partial f}{\partial \theta^{(t)}}$
5: Update your parameter by subtracting the (scaled) derivative

$$\theta^{(t+1)} \leftarrow \theta^{(t)} - \eta \frac{\partial f}{\partial \theta^{(t)}}$$

- η is the **step size** or **learning rate**, a parameter
- When to stop? Fix a number of iterations, or define other criteria



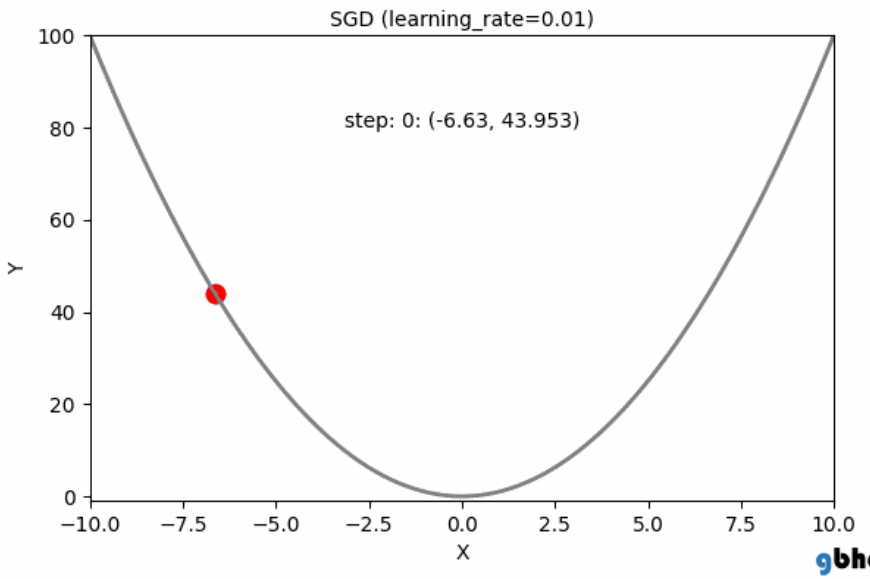
Learning rate

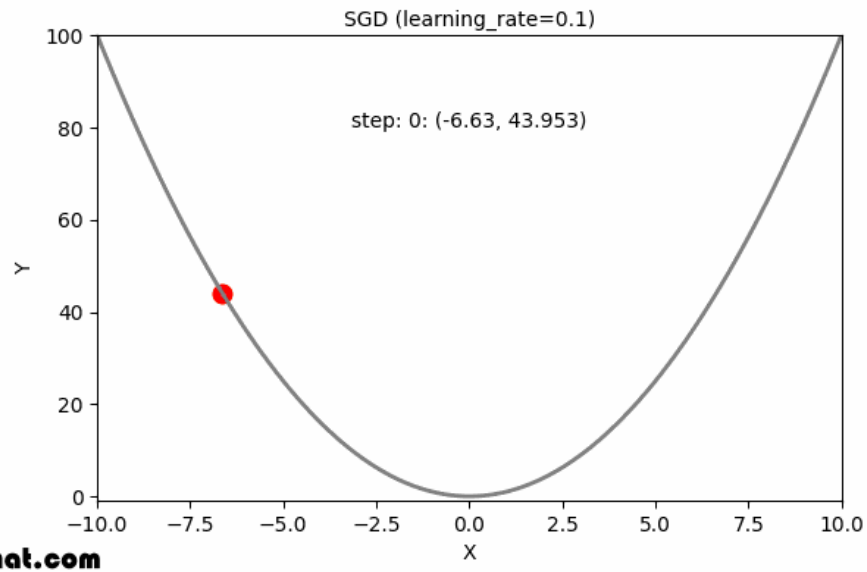


- η is learning rate, which defines how big a step to update the θ
  - If η is too small, the algorithm might be slow.
  - If η is too large, you might miss the minimum.

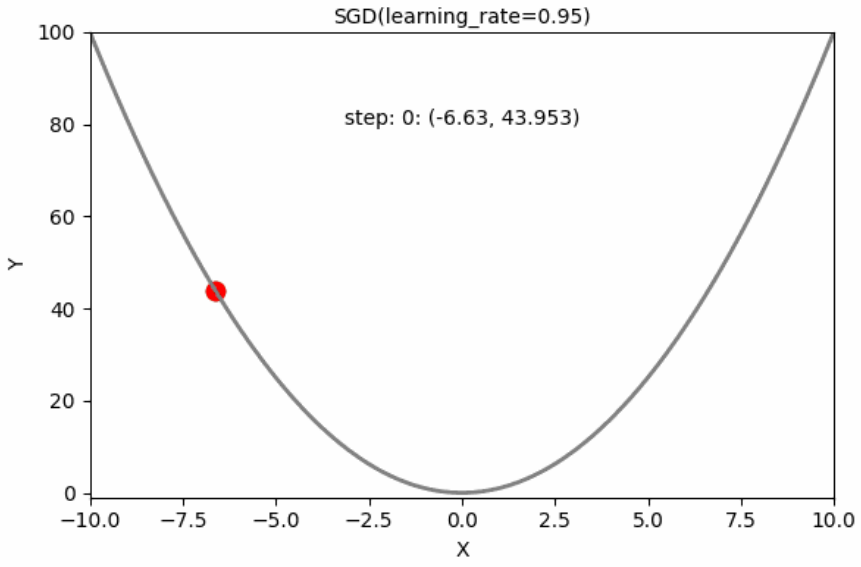
THE UNIVERSITY OF MELBOURNE

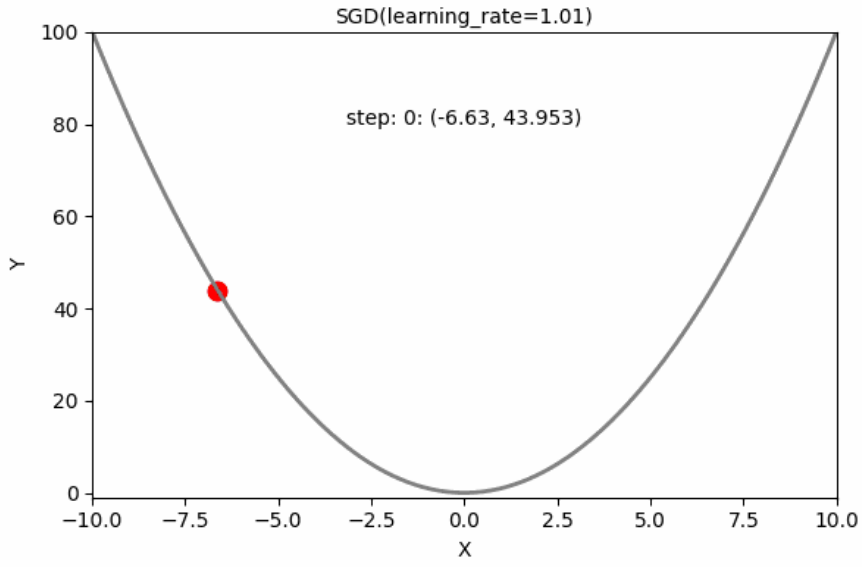
Learning rate





9bhat.com





Gradient Descent: Algorithm

Recipe for Gradient Descent (multiple parameters)

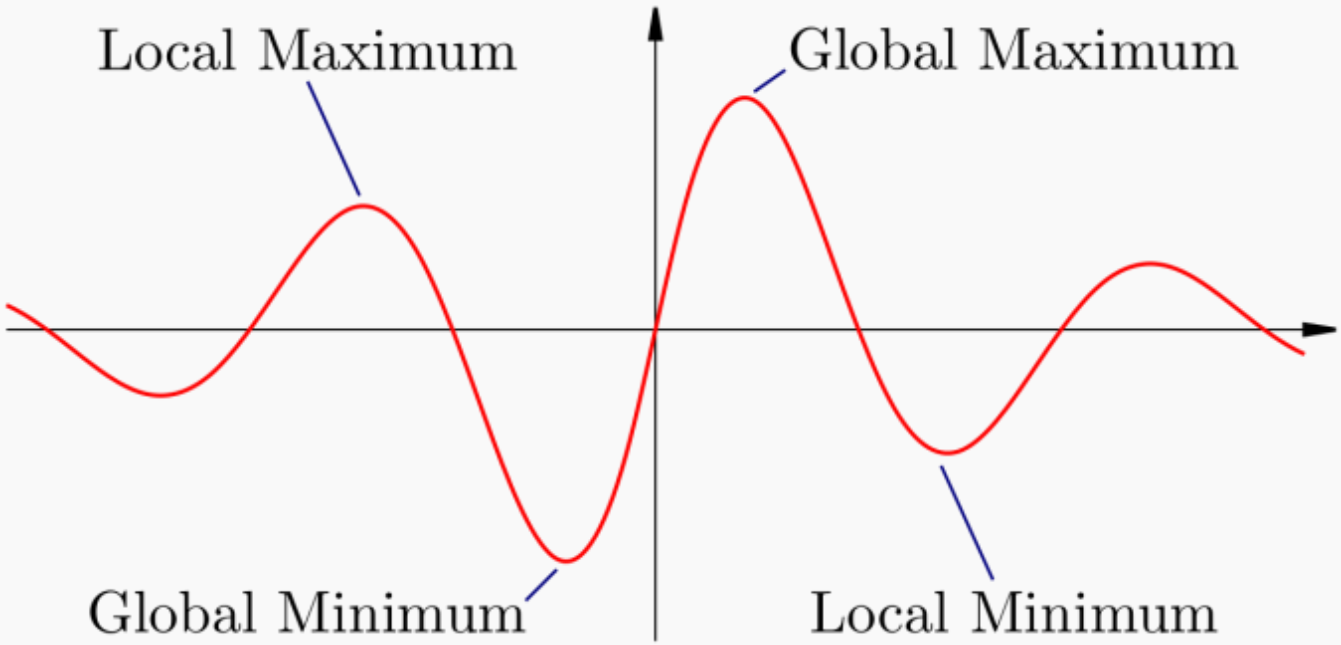
1: Define objective function $f(\theta)$
2: Initialize parameters $\{\theta_1^{(0)}, \theta_2^{(0)}, \theta_3^{(0)}, \dots\}$
3: **for** iteration $t \in \{0, 1, 2, \dots T\}$ **do**
4: Initialize vector of *gradients* $\leftarrow$ []
5: **for** parameter $f \in \{1, 2, 3, \dots F\}$ **do**
6: Compute the first derivative of f at that point $\theta_f^{(t)}$: $\frac{\partial f}{\partial \theta_f^{(t)}}$
7: append $\frac{\partial f}{\partial \theta_f^{(t)}}$ to *gradients*
8: **Update all** parameters by subtracting the (scaled) gradient

$$\theta^{(t+1)} \leftarrow \theta^{(t)} - \eta \frac{\partial f}{\partial \theta^{(t)}}$$

Aside: Global and Local Minima and Maxima

Possible issue: local maxima and minima!

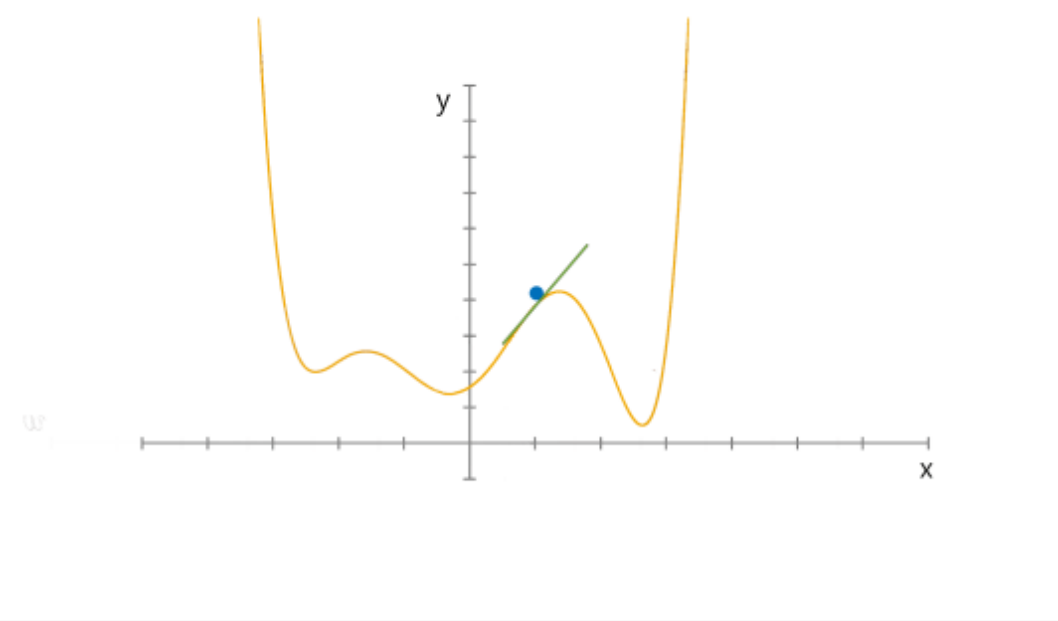
- A function is **convex** if a line between any two points of the function lies above the function
- A global **maximum** is the single highest value of the function
- A global **minimum** is the single lowest value of the function



https://commons.wikimedia.org/wiki/File:Maxima_and_Minima.svg

Gradient Descent Guarantees

1. with an appropriate learning rate, GD will find the global minimum for differentiable convex functions
2. with an appropriate learning rate, GD will find a local minimum for differentiable non-convex functions



Quiz

pollev.com/iml2025



THE UNIVERSITY OF MELBOURNE

Logistic Regression

Quick Refresher

Recall **Naive Bayes**

$$P(x, y) = P(y)P(x|y) \quad = \prod_{i=1}^{N} P(y^i) \prod_{m=1}^{M} P(x_m^i|y^i)$$

- a **probabilistic generative model** of the joint probability $P(x, y)$
- optimized to maximize the likelihood of the observed data
- **naive** due to unrealistic feature indepencence assumptions

For **prediction**, we apply **Bayes Rule** to obtain the conditional distribution

$$P(x, y) = P(y)P(x|y) = P(y|x)P(x)$$

$$P(y|x) = \frac{P(y)P(x|y)}{P(x)}$$

$$\hat{y} = \underset{y}{\text{argmax}}\, P(y|x) \approx P(y)P(x|y)$$

Prior probability

Likelihood

How about we model $P(y|x)$ directly? → **Logistic Regression**



Logistic Regression on a high level

- Is a **binary** classification model
- Is a **probabilistic discriminative model** because it optimizes $P(y|x)$ directly
- Learns to optimally discriminate between features that belong to different classes
- No model of Likelihoods ($P(x|y)$) → no conditional feature independence assumption



Logistic Regression: Derivation I

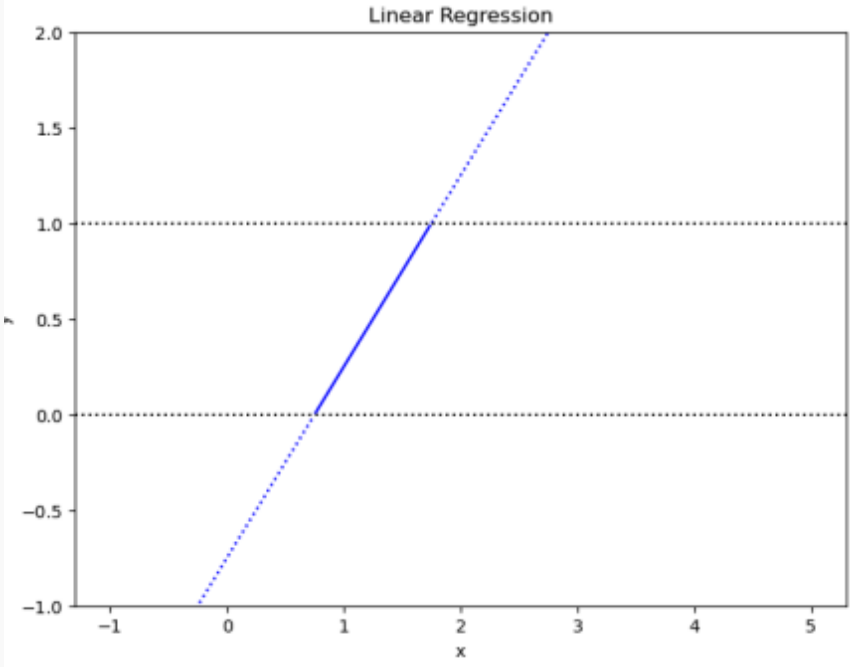
- Let's assume a **binary** classification task, y is true (1) or false(0).
- We model **probabilities** $P(y = 1 | x; \theta) = p(x)$ as a function of observations x under parameters θ. [What about $P(y = 0 | x; \theta)$?]
- We want to use a **regression** approach



Logistic Regression: Derivation I

- Let's assume a **binary** classification task, y is true(1) or false(0).
- We model **probabilities** $P(y=1|x;\theta)=p(x)$ as a function of observations x under parameters θ. [What about $P(y=0|x;\theta)$?]
- We want to use a **regression** approach
- How about: $p(x)$ as a linear function of x. $p(x)=\theta_0+\theta_1 x_1+\ldots\theta_F x_F$

Problem:
probabilities are NOT bounded between 0 and 1



Logistic Regression: Derivation I

- Let's assume a **binary** classification task, y is true(1) or false(0).
- We model **probabilities** $P(y=1|x;\theta)=p(x)$ as a function of observations x under parameters θ. [What about $P(y=0|x;\theta)$?]
- We want to use a **regression** approach

~~• How about: $p(x)$ as a linear function of x.~~ $p(x)=\theta_0+\theta_1 x_1+\ldots\theta_F x_F$

—

We need to modify p(x) such that it is bounded between 0 and 1.

Logistic Regression: Derivation I

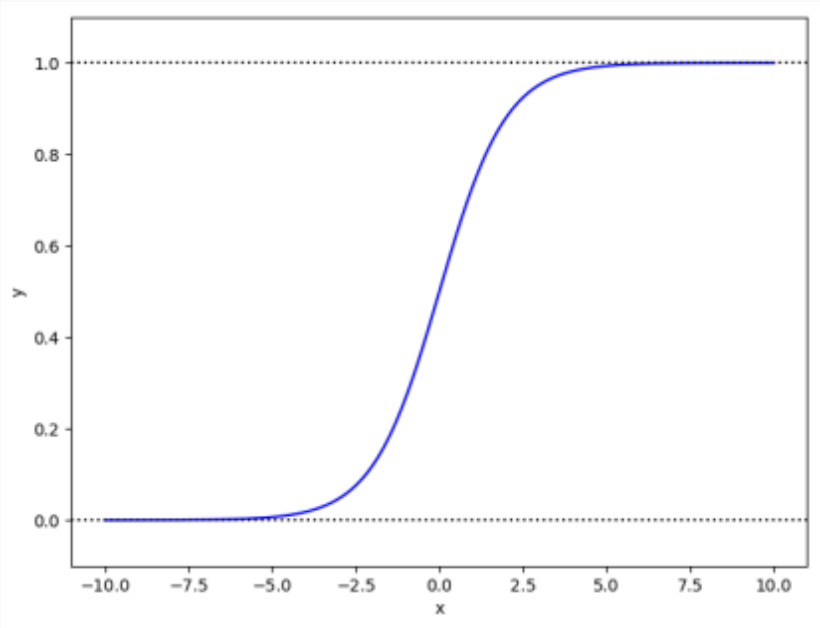
- Let's assume a **binary** classification task, y is true(1) or false(0).
- We model **probabilities** $P(y=1|x;\theta)=p(x)$ as a function of observations x under parameters θ. [What about $P(y=0|x;\theta)$?]
- We want to use a **regression** approach

~~• How about: $p(x)$ as a linear function of x.~~ $p(x)=\theta_0+\theta_1 x_1+\ldots\theta_F x_F$

—

We need to modify p(x) such that it is bounded between 0 and 1.
We will model the "log odds"

$$log\frac{p(x)}{1-p(x)}=\theta_0+\theta_1 x_1+\ldots\theta_F x_F$$



Odds vs Probability

- Both odds and probability are ways to express the likelihood of an event occurring
  - **Probability**: measure the chance of an event
  - **Odds**: compare the likelihood of the event happening versus not happening.



$$P('red') = \frac{2}{5}$$

$$Odds('red') = \frac{2}{3} < 1 \qquad Odds(NOT\ 'red') = \frac{3}{2} > 1$$

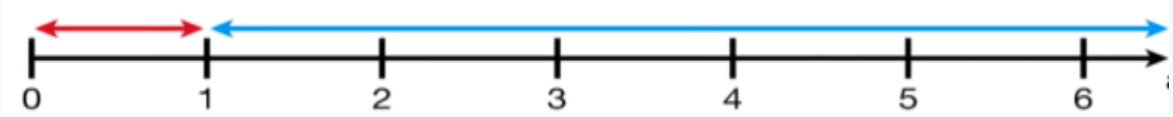
Odds are **against** 'red'

$$Odds = \frac{P(x)}{1-P(x)}$$

$$odds('red') = \frac{P('red')}{1 - P('red')} = \frac{\frac{2}{5}}{1 - \frac{2}{5}} = \frac{\frac{2}{5}}{\frac{3}{5}} = \frac{2}{3}$$

Odds function

- Odds against something: the probability of it happening (nominator) is less than the probability of it *not* happening (denominator)
  $0 < odds < 1$
- Odds in favour of something: the probability of it happening is bigger than the probability of it *not* happening
  $odds > 1$

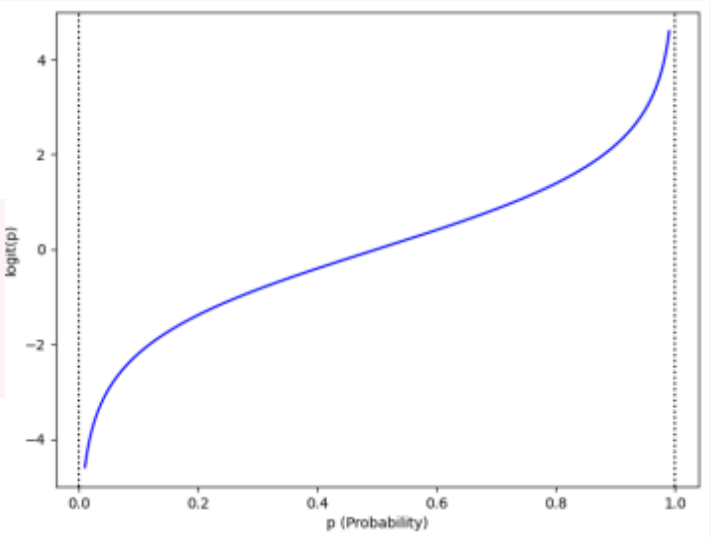


- solve this problem by log function $(log\ odds)$



Logit function

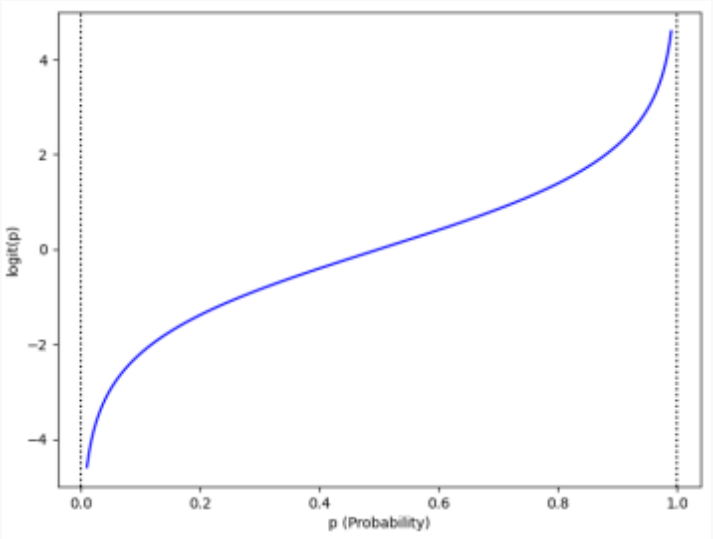
$$log\frac{p(x)}{1-p(x)} = \theta_0 + \theta_1 x_1 + ...\theta_F x_F$$



THE UNIVERSITY OF MELBOURNE

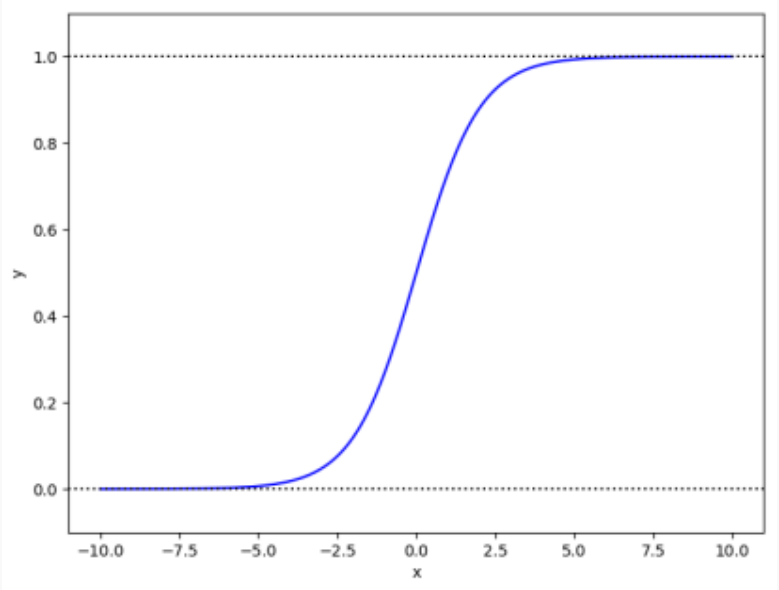
Find the probability using the odds

$$log\frac{p(x)}{1-p(x)} = \theta_0 + \theta_1 x_1 + \ldots \theta_F x_F$$



$$P(x) = \frac{1}{1 + e^{-\boldsymbol{\theta}\cdot\boldsymbol{x}}}$$

Logistic or **sigmoid (σ)** function



Regressing the Logit function

$$\text{logit}\, P(\text{x}) = \log\frac{P(x)}{1-P(x)} = \theta_0 + \theta_1 x_1 + \cdots + \theta_F x_F = \boldsymbol{\theta} \cdot \boldsymbol{x}$$

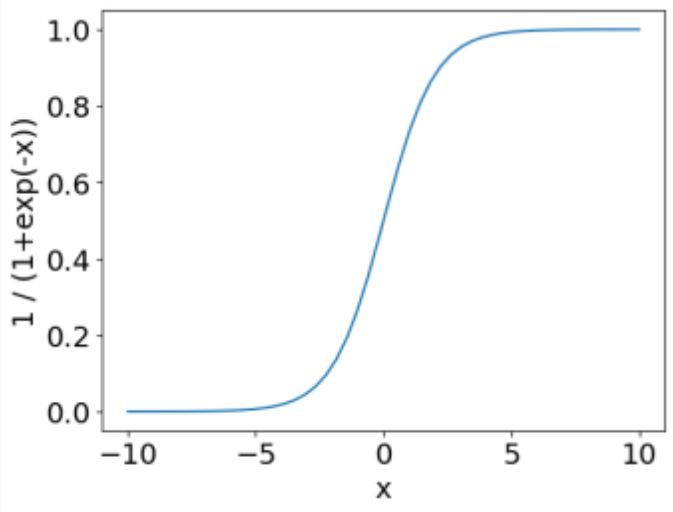
If we rearrange and solve for *P*(*x*), we get

$$P(x) = \frac{1}{1 + e^{-\boldsymbol{\theta}\cdot\boldsymbol{x}}}$$

$$-\infty < x < +\infty$$

$$0 < P(x) < 1$$

- we pass a regression model through the logistic function to obtain a valid probability prediction



Logistic Regression: Interpretation

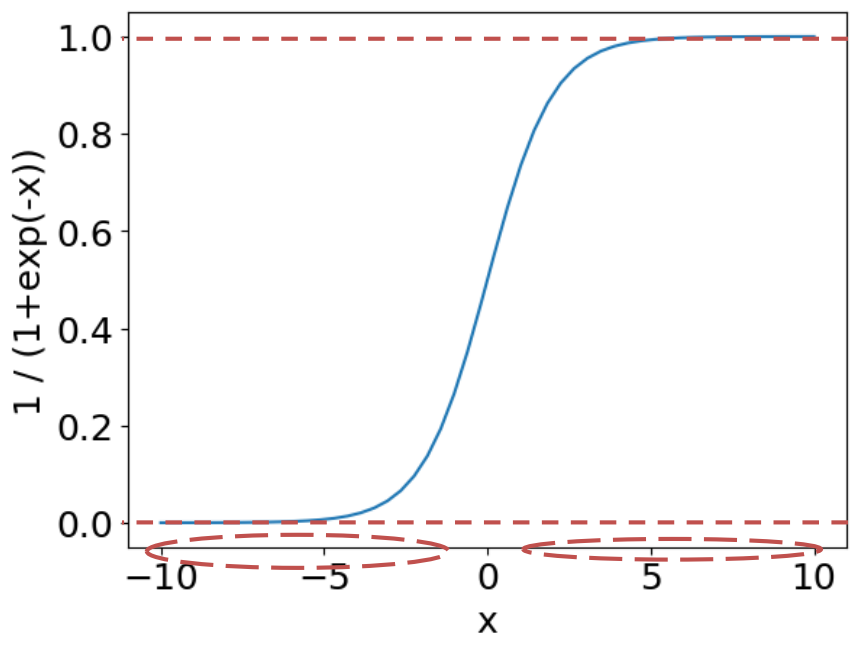
$$P(y = 1|x;\theta) = \frac{1}{1 + e^{-\theta \cdot X}}$$

A closer look at the logistic function

Most inputs lead to $P(y/x)=0$ or $P(y/x)=1$. That is intended because all true labels are either 0 or 1.

$$\theta \cdot X = \theta_0 + \theta_1 x_1 + \cdots + \theta_F x_F = \theta_0 + \sum_{f=1}^{F} \theta_f x_f$$

- $\theta \cdot X > 0$ means $y = 1$
- $\theta \cdot X \approx 0$ means $y\ is\ uncertain$
- $\theta \cdot X < 0$ means $y = 0$

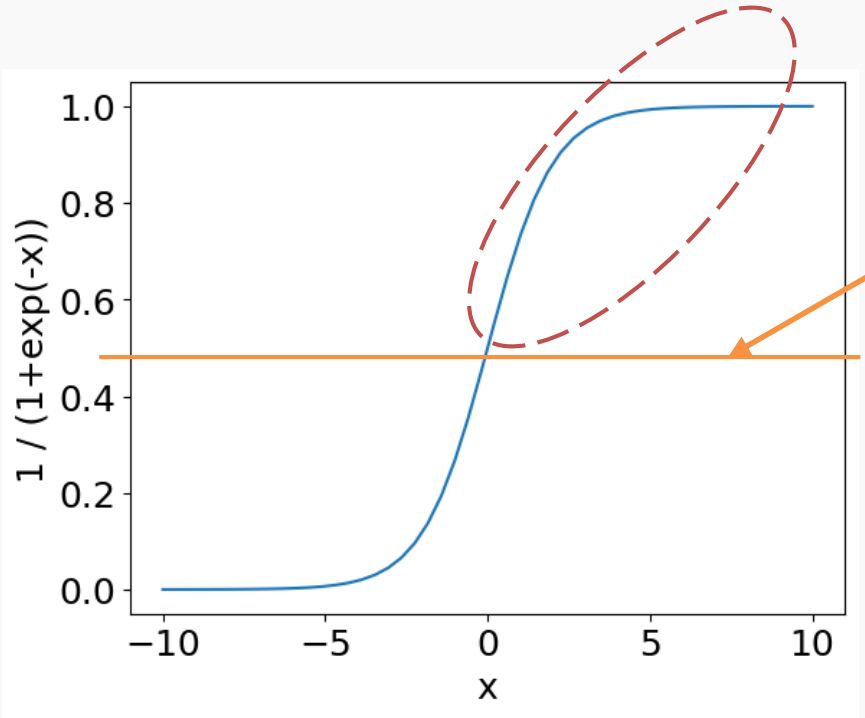


Logistic Regression: Prediction

- The logistic function returns the probability of $P(y = 1)$ given an in put x

$$P(y = 1|x; \theta) = \frac{1}{1 + e^{-\theta \cdot x}} = \sigma(x; \theta)$$

- We define a **decision boundary**, e.g., predict $y = 1$ if $P(y = 1|x; \theta) > 0.5$ and $y = 0$ otherwise



Logistic Regression: Example

ID	Age	Edu-num	Hrs/wk	Income
1	31	13	40	>50K
2	31	9	20	<=50K
3	25	13	60	>50K
4	70	9	40	>50K
5	39	9	45	<=50K



Logistic Regression: Example

ID	Age	Edu-num	Hrs/wk	Income
1	31	13	40	1
2	31	9	20	0
3	25	13	60	1
4	70	9	40	1
5	39	9	45	0

$$P(y = 1|x;\theta) = \frac{1}{1 + e^{-\theta \cdot X}}$$

$$= \frac{1}{1 + e^{-\theta \cdot X}} = \frac{1}{1 + e^{-(\theta_0 + \theta_1 \times Age + \theta_2 \times Edu_{num} + \theta_3 \times Hrs_{wk})}}$$

$$= \sigma(\theta_0 + \theta_1 \times Age + \theta_2 \times Edu_{num} + \theta_3 \times Hrs_{wk})$$

Logistic Regression: Example

$$P(y = 1|x; \theta) = \frac{1}{1 + e^{-(\theta_0 + \theta_1 \times Age + \theta_2 \times Edu_{num} + \theta_3 \times Hrs_{wk})}}$$

Let's consider θ: $(-18, 0.12, 0.96, 0.1)$

What is the label for this test data?

ID	Age	Edu-num	Hrs/wk	Income
6	20	10	25	?

Testing the model

$$P(y = 1|x;\theta) = \frac{1}{1 + e^{-(\theta_0 + \theta_1 \times Age + \theta_2 \times Edu_{num} + \theta_3 \times Hrs_{wk})}}$$

θ: $(-18, 0.12, 0.96, 0.1)$

ID	Age	Edu-num	Hrs/wk	Income
6	20	10	25	?

$$P(y = 1|x;\theta) = \frac{1}{1 + e^{-(-18 + 0.12 \times 20 + 0.96 \times 10 + 0.1 \times 25)}} = \sigma(3.5) \approx 0.03$$ <0.5

Predict y=1 if p(y=1|x;θ) > 0.5 and y=0 otherwise

Prediction: y = 0. The predicted class is <=50k

Training the model

But how do we find the θ?

- We need to define an objective function to optimise.

-> Minimise the Loss function

- Given *N* independent training instances $\{\boldsymbol{X}, Y\}$, we want to choose $\boldsymbol{\theta}$ to *minimise the* ***negative*** *likelihood* of observing them

$$\mathcal{L}(\beta) = -\,\mathrm{P}(Y|\boldsymbol{X}, \boldsymbol{\theta}) = -\prod_{i=1}^{N} \mathrm{P}(y_i|\boldsymbol{x}_i, \boldsymbol{\theta})$$

Subject to,

$$P(y = 1|x; \theta) = \sigma(\theta \cdot x)$$

$$P(y = 0|x; \theta) = 1 - \sigma(\theta \cdot x)$$



Minimising the Loss Function

- Given *N* independent training instances $\{X, Y\}$, we want to choose θ to *minimise the negative likelihood* of observing them

$$\mathcal{L}(\beta) = -\mathrm{P}(Y|\boldsymbol{X}, \boldsymbol{\theta}) = -\prod_{i=1}^{N} \mathrm{P}(y_i|\boldsymbol{x}_i, \boldsymbol{\theta})$$

$$= -\prod_{i=1}^{N} (\sigma(\boldsymbol{\theta} \cdot \boldsymbol{x}_i))^{y_i} (1 - \sigma(\boldsymbol{\theta} \cdot \boldsymbol{x}_i))^{1-y_i}$$

$$\widehat{\boldsymbol{\theta}} = \arg\min_{\theta} \left(-\prod_{i=1}^{N} \left(\sigma(\boldsymbol{\theta} \cdot \boldsymbol{x}_i)\right)^{y_i} \left(1 - \sigma(\boldsymbol{\theta} \cdot \boldsymbol{x}_i)\right)^{1-y_i}\right)$$

Take a log

$$\log(\mathcal{L}(\theta)) = \sum_{i=1}^{N} y_i \log\left(\sigma(\boldsymbol{\theta} \cdot \boldsymbol{x}_i)\right) + (1 - y_i) \log(1 - \sigma(\boldsymbol{\theta} \cdot \boldsymbol{x}_i))$$



Take 1st derivative of the Loss Function

- Calculate the derivatives

$$\frac{\partial \log(\mathcal{L}(\theta))}{\partial \theta_j} = \sum_{i=1}^{N} (\sigma(\theta \cdot x_i) - y_i) x_{ij}$$

- Now we need to set the derivative equal to Zero and sole it for θ
- Unfortunately, the problem is not analytically solvable
- Instead, we'll use the iterative model, Gradient Descent

$$\theta^{t+1} = \theta^t - \eta \frac{\partial f}{\partial \theta^t}$$

$$\theta_j^{t+1} = \theta_j^t - \eta \sum_{i=1}^{F} (\hat{y}_i - y_i) x_{ij}$$

THE UNIVERSITY OF MELBOURNE

Logistic Regression is a binary classifier

- Logistic regression uses the logistic function (or sigmoid function) to model the probability that a given input belongs to a particular class (y=1).

$$\sigma(y = 1|x, \theta) = \frac{1}{1 + e^{-\theta.x}} = \frac{e^{\theta.x}}{e^{\theta.x} + 1}$$

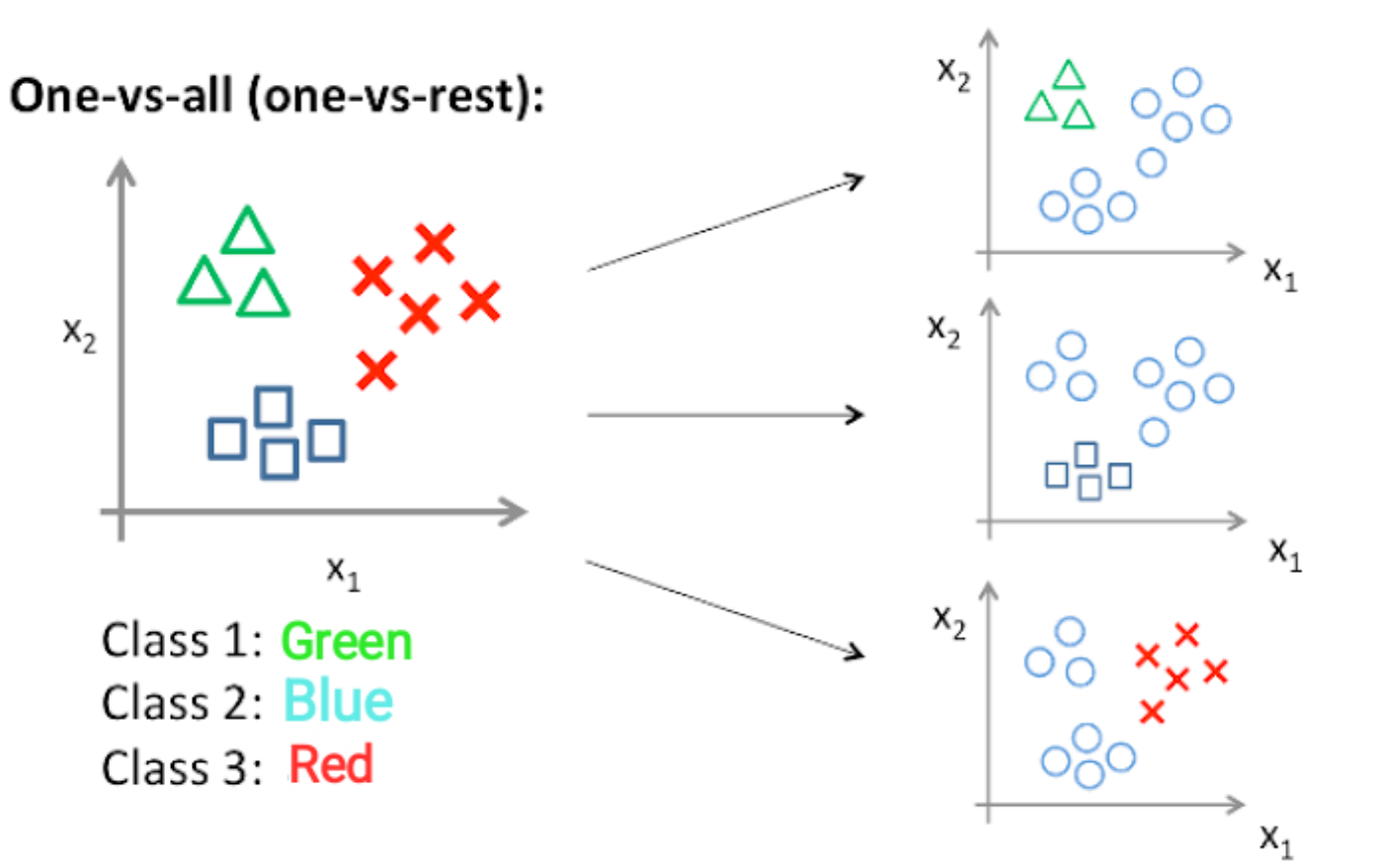
- It predicts the probability that the output belongs to the positive class.
- If the predicted probability is greater than 0.5, the output is 1 (positive class); otherwise, it is classified as 0 (negative class).
- During training, logistic regression learns the weights for each feature $(\theta_0, \theta_1, \ldots, \theta_f)$

THE UNIVERSITY OF MELBOURNE

Using Logistic Regression in a Multiclass Classification

1. One-vs-all (One-vs-rest) Classification

In this approach, we train a separate binary logistic regression classifier for each class. Each classifier distinguishes one class from all other classes.



Using Logistic Regression in a Multiclass Classification

2. Softmax Regression (Multinomial Logistic Regression)

Softmax regression is an extension of logistic regression for multiclass classification. It generalizes the binary logistic regression model to handle multiple classes simultaneously.

$$\sigma(y = j|x, \theta) = \frac{e^{\theta_j . x}}{\sum_{i=1}^{f} e^{\theta_i . x}}$$

THE UNIVERSITY OF MELBOURNE

Softmax Example

- Suppose we have a classification problem with 3 classes: A, B, and C. Given an input feature vector x, we want to compute the probabilities for each class using softmax regression.

- For simplicity, let's assume we have the following weights for each class:
  - Class A: $\theta_A = [0.1, 0.5, -0.2]$
  - Class B: $\theta_B = [-0.1, -0.3, 0.4]$
  - Class C: $\theta_C = [0, 0.1, 0.2]$

- Our input (test) instance is x= [1,2]

1. We first calculate the Logit for each class:
   - Class A: $\theta_A . x = [0.1, 0.5, -0.2].[1,1,2] = 0.2$
   - Class B: $\theta_B . x = [-0.1, -0.3, 0.4].[1,1,2] = 0.4$
   - Class C: $\theta_C . x = [0, 0.1, 0.2].[1,1,2] = 0.5$

THE UNIVERSITY OF MELBOURNE

Softmax Example (Cont.)

2. Then we compute the exponential scores:
   - Class A: $\theta_A . x = e^{0.2} \approx 1.221$
   - Class B: $\theta_B . x = e^{0.4} \approx 1.491$
   - Class C: $\theta_C . x = e^{0.5} \approx 1.649$

3. Finaly we compute the softmax probabilities:
   - $P(y = A|x) = \frac{1.221}{1.221+1.491+1.649} \approx 0.28$
   - $P(y = B|x) = \frac{1.491}{1.221+1.491+1.649} \approx 0.342$
   - $P(y = A|x) = \frac{1.649}{1.221+1.491+1.649} \approx 0.378$

The class with the highest probability is Class C, so the model would predict Class C for this input.



Discriminative vs Generative Models

Naïve Bayes: Generative Model

- learns the joint distribution of features and their classes
  p(features, dog)
  p(features, cat)
- could be used to generate new class members



Logistic Regression: Discriminative Model

- does **not** learn feature – class connections
- only learns the boundary between classes, but nothing about the classes themselves
  p(cat | features)
  p(dog| features)

THE UNIVERSITY OF MELBOURNE

Logistic Regression: Final Thoughts

Pros

- Probabilistic interpretation
- No restrictive assumptions on features
- Often outperforms Naive Bayes
- Particularly suited to frequency-based features

Cons

- Can only learn *linear* feature-data relationships
- Some feature scaling issues
- Often needs a lot of data to work well
- Regularisation a nuisance, but important since overfitting can be a big problem



References

Cosma Shalizi. *Advanced Data Analysis from an Elementary Point of View*. Chapters 11.1 and 11.2. Online Draft.
https://www.stat.cmu.edu/~cshalizi/ADAfaEPoV/ADAfaEPoV.pdf

Dan Jurafsky and James H. Martin. *Speech and Language Processing*. Chapter 5. Online Draft V3.0. https://web.stanford.edu/~jurafsky/slp3/